$$\theta_0 = \theta_0 - \alpha \sum_{i=1}^{n} (h_\theta(x^{(i)}) - y^{(i)})$$

$$\theta_1 = \theta_1 - \alpha \sum_{i=1}^{n} (h_\theta(x^{(i)}) - y^{(i)})\, x_1^{(i)}$$

$$\theta_2 = \theta_2 - \alpha \sum_{i=1}^{n} (h_\theta(x^{(i)}) - y^{(i)})\, x_2^{(i)}$$

for (compu... it has to tak... all n samp... computationally expensive

Numerical Example

x_0	x_1	x_2	y
1	1	2	3
1	2	1	4
1	3	3	5

$n = 3$ $\quad d = 2$

$\theta_0, \theta_1, \theta_2 = 0$ $\quad \alpha = 0.1$

$h = \theta_0 x_0 + \theta_1 x_1 + \theta_2 x_2$

Apply gradient descent and compute new θ value after 2 iterations

$h_\theta(x^{(i)}) = \theta_0 x_0 + \theta_1 x_1 + \theta_2 x_2 + \cancel{\theta_3 = 3}$

Initially $\Rightarrow h_\theta(x) = 0$

For θ_0 and $n = 1$

$\theta_0 = \cancel{0 - (0.1)}\,(0 - 3)$

$\theta_0 = \cancel{0 - (0.1)}\,(-3)$

$\theta_0 = \cancel{0.3}\ -3$

For θ_0 and $n = 2$

$= (0 - 4)\,1$

$= -4$

For θ_0 and $n = 3$

$= (0 - 5)$

$= -5$

Sum $= -3 + (-4) + (-5)$

$= -12$

$\theta_0 = 0 - 12$

$\theta_0 = -12\,(0.1)$

$\boxed{\theta_0 = -1.2}$

for θ_1 and $n=1$

$= (0-3)1$

$= -3$

for θ_1 and $n=2$

$= (0-4)2$

$= -8$

for θ_1 and $n=3$

$= (0-5)3$

$= -15$

$\theta_1 = 0 - (-3-8-15)$

$\theta_1 = 26(0.1)$

$\boxed{\theta_1 = 2.6}$

for θ_2 and $n=1$

$= (0-3)2$

$= -6$

for θ_2 and $n=2$

$= (0-4)1$

$= -4$

for θ_2 and $n=3$

$= (0-5)3$

$= -15$

$\boxed{\theta_2 = 2.5}$

At end of 1st iteration

$h_\theta(x^1) = 1.2 + 2.6(1) + (2.5)2$

$h_\theta(x^1) = 8.8$

$h_\theta(x^2) = 1.2 + 2.6(2) + 2.5$

$= 1.2 + 5.2 + 2.5$

$= 8.9$

$h_\theta(x^3) = 1.2 + 7.8 + 7.5$

$= 16.5$

2nd Iteration

for $\theta_0 = 1.2$ and $n=1$

$= (8.8 - 3)$

$= 5.8$

for $\theta_0 = 1.2$ and $n=2$

$= (8.8 - 4)$

$= 4.8$

for $\theta_0 = 1.2$ and $n=3$

$= (8.8 - 5)$

updated $\theta_0 = 1.44 - 1.2$

$\boxed{\theta_0 = 0.24}$

$= (16.5 - 3)2$

$= 27$

$= 12.5$

sum $= 2.5 - 7.4$

$\boxed{\theta_2 = -4.9}$

$h_{\theta_0}(x) = 0.24 + (-0.44)$

2nd Iteration for $n=1$ $y' = 3$ $x'_0 = 1$

$\boxed{h_\theta(x') = 8.8}$ — 3

for $n=1 \Rightarrow (8.88 - 3)1 = 5.8$

for $n=2 \Rightarrow (8.9 - 4)1 = 4.9$

for $n=3 \Rightarrow (16.5 - 5)1 = 11.5$

22.2

$\theta_0 = 1.2 - 2.22$

$\boxed{\theta_0 = -1.02}$

for $h_\theta x^1 = 8.8$, $h_\theta x^2 = 8.9$, $h_\theta x^3 = 16.5$

$n=1 \Rightarrow (8.8-3)1 = 5.8$

$n=2 \Rightarrow (8.9-4)2 = 9.8$

$n=3 \Rightarrow (16.5-5)3 = 34.5$

$\overline{5\ 0.1}$

$\begin{array}{r} 5.01 \\ 2.6 \\ \hline 2.41 \end{array}$

$\theta_1 = 2.6 - 5.01$

$\boxed{\theta_1 = -2.41}$

for

$n=1 \Rightarrow (8.8-3)2 = 11.6$

$n=2 \Rightarrow (8.9-4)1 = 4.9$

$n=3 \Rightarrow (16.5-5)3 = 34.5$

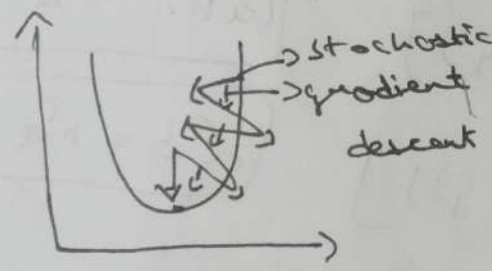
$\overline{5\ 1.0}$

$\begin{array}{r} 5.1 \\ 2.5 \\ \hline 2.6 \end{array}$

$\theta_2 = 2.5 - 5.1$

$\boxed{\theta_2 = -2.6}$

Stochastic gradient descent (very effective, large n)



for each features (x_d) instead of 1 to n, random samples are picked and θ calculated until convergence.

$\theta_j := \theta_j - \alpha\left[(h_\theta x^i - y^i)x_j^i\right]$ ⇒ for every j

... algorithm

$$X = \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}$$

$$y = \begin{pmatrix} 5 \\ 11 \end{pmatrix}$$

$$X^T = \begin{pmatrix} 1 & 3 \\ 2 & 4 \end{pmatrix} \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}$$

$$X^T X = \begin{pmatrix} 10 & 14 \\ 14 & 20 \end{pmatrix}$$

$$= \frac{1}{4} \begin{pmatrix} 20 & -14 \\ -14 & 10 \end{pmatrix}$$

$$= \begin{pmatrix} 5 & -\frac{14}{4} \\ -\frac{14}{4} & \frac{10}{4} \end{pmatrix} \begin{pmatrix} 1 & 3 \\ 2 & 4 \end{pmatrix}$$

$$= \begin{pmatrix} 5-7 & 15-14 \\ -\frac{14}{4}+5 & -\frac{42}{4}+10 \end{pmatrix}$$

$$= \begin{pmatrix} -2 & 1 \\ \frac{6}{4} & -\frac{2}{4} \end{pmatrix}$$

$$= \begin{pmatrix} -2 & 1 \\ 3/2 & -\frac{1}{2} \end{pmatrix} \begin{pmatrix} 5 \\ 11 \end{pmatrix}$$

$$= \begin{pmatrix} 1 \\ \frac{15}{2} - \frac{11}{2} \end{pmatrix} = \begin{pmatrix} 1 \\ 2 \end{pmatrix}$$

$$= -\left(y - g(\theta^T x)\right) x_j$$

$$= \left(y - h_\theta(x)\right) x_j$$

$$\boxed{\theta := \theta - \alpha \left(y - h_\theta(x)\right) x_j}$$

convex function, global minima or maxima

similar for linear regression

only change is $h_\theta(x)$

(-) change due to gradient ascent and descent

Linear and logistic regression are Generalized linear models (GLM).

$$\boxed{\theta_j := \theta_j + \alpha \sum_{i=1}^{n} \left(y^{(i)} - h_\theta(x^{(i)})\right) x_j^{(i)}}$$ for every j

Example:

x_1	x_2	y
1	0	0
2	1	0
3	1	0
4	2	1
5	3	1

$\theta_1 = 0$, $\theta_2 = 0$, $\alpha = 0.1$

$h_\theta(x) = g(\theta_1 x_1 + \theta_2 x_2)$

$h_\theta(x) = g(0)$ for all 5 samples

$g(0) = \dfrac{1}{1 + e^0} = 0.5$ for all 5 samples

$h_\theta(x) = 0.5$

for $\theta_1 = \theta_1 + \alpha \sum_{i=1}^{n} \left(y^{(i)} - h_\theta(x^{(i)})\right) x_1^{(i)}$

for $n = 1$

$= (0 - 0.5)1 + (0 - 0.5)2 + (0 - 0.5)3 + (1 - 0.5)4 + (1 - 0.5)5$

$= -0.5 - 1 - 1.5 + 2 + 2.5$

$= 1.5 \times \alpha = 0.15$

$\theta_1 = 0 + 0.15 = 0.15$

$\theta_2 = 0 + 0.15 = 0.15$

for $\theta_2 = \theta_2 + \alpha \sum_{i=1}^{n} \left(y^{(i)} - h_\theta(x^{(i)})\right) x_2^{(i)}$

$\boxed{\begin{array}{l}\theta_1 = 0.15 \\ \theta_2 = 0.15\end{array}}$

$= (0 - 0.5)0 + (0 - 0.5)1 + (0 - 0.5)1 + (1 - 0.5)2 + (1 - 0.5)3$

$= 0 - 0.5 - 0.5 + 1 + 1.5$

In ... tree

Instead of squared error

1) Binary cross entropy loss
2) Gini Index
3) Information Gain (opp Entropy)

training term (1) $E(T) = \sum_{i=1}^{n} E\left(T\left(x^{(i)}, y^{(i)}\right)\right) + \boxed{\lambda |T|}$ T is number of leaves

penalty term

Similar regularization term in linear regression like l_1 and l_2 norm.

Depth of tree comes smaller when there are less number of leaves, so number of leaves must be reduced (penalty term).

If λ is more than underfitting may happen

Exercise

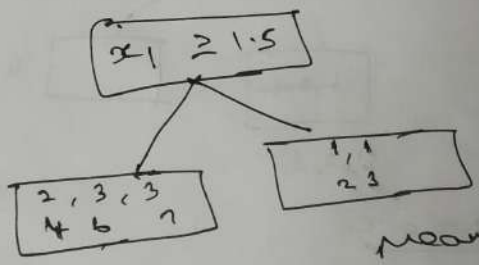
$K=2$

Build Tree(S, 2)

Sno	x_1	x_2	y
1	1	1	2
2	1	2	3
3	2	2	4
4	3	2	6
5	3	3	7

for $x_1 = (1, 2, 3)$

$S = (1.5, 2.5)$



Mean $= \frac{2+3}{2}$

$= 2.5$

$= (2-2.5)^2 + (3-2.5)^2$

$= 0.25 + 0.25$

$= 0.5$

Mean $= \frac{4+6+7}{3}$

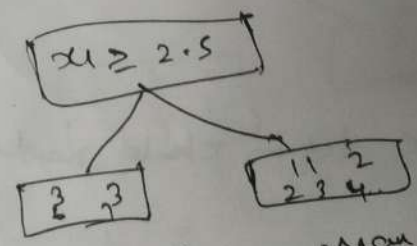
$= 5.6$

$= (5.6-4)^2 + (6-5.6)^2 + (7-5.6)^2$

$= 2.56 + 0.16 + 1.96$

$= 4.68$

j	S_j	E
x_1	1.5	5.18
	2.5	2.5
x_2	1.5	10



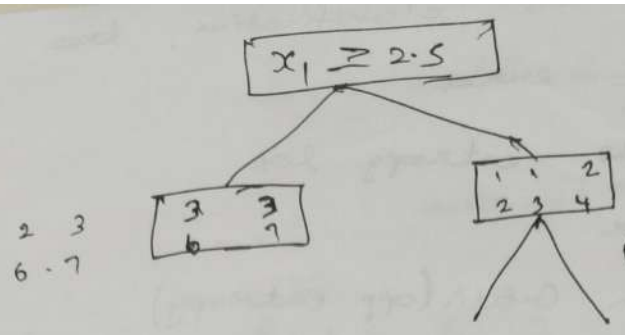
Mean $= 6.5$

$= 0.5$

Mean $= 3$

$= 1 + 0 + 1$

$= 2.5$ $[2 + 0.5]$



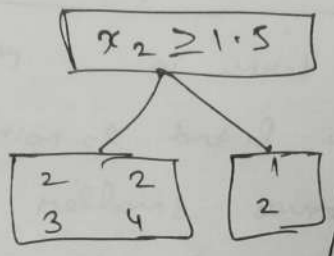
2 3
6 7

1 2 2
2 3 4

Build Tree

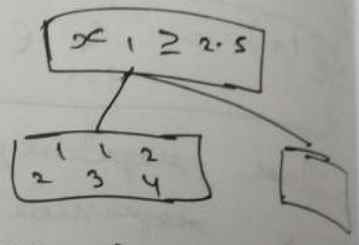
	S.val	Val
x_1	2.5	2
	3.5	2.75
x_2	1.5	0.5

$x_2 = (1, 2, 2)$



mean = 3.5

E = 0.5



mean = 3

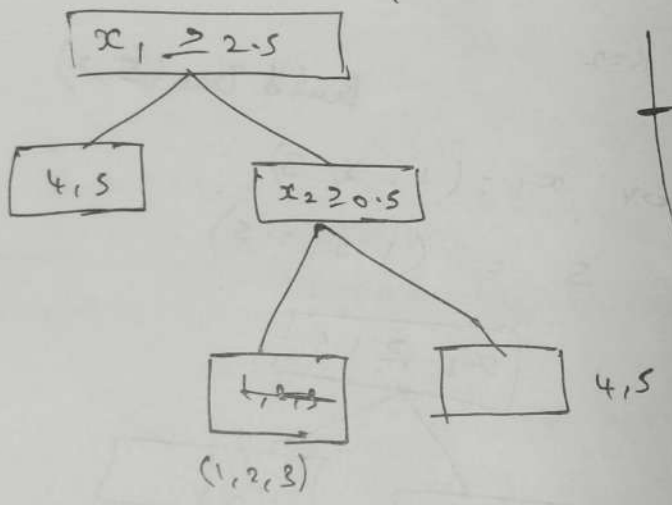
E = 1.41

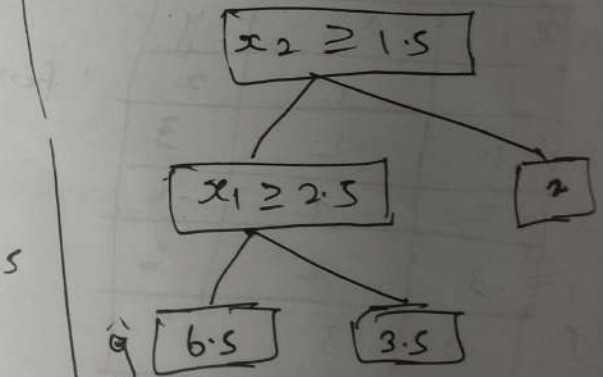
= 2

$(1.5)^2 + (0.5)^2 (0.5)^2$

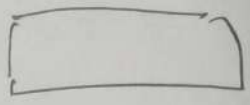
= 2.25 + 0.25 + 0.25

= 2.75





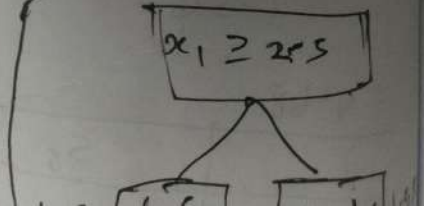
Root node

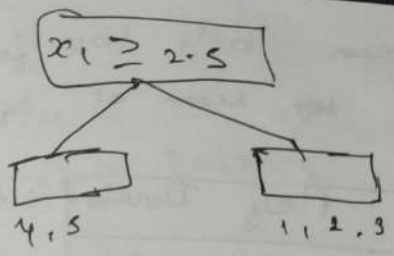


(i) for Root node, check x_1, x_2

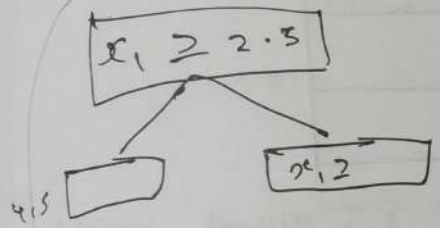
for x_1, use cand. split = $\{1.5, 2.5\}$

(ii) for left child node, check x_1, x_2

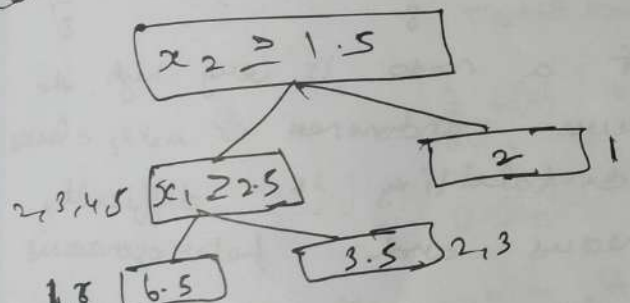




x_1	1	1	2
x_2	1	2	2
y	2	3	4



Correct.



$x_1 = 1 \quad 2 \quad 3 \quad 3$

S.U = 1.5 , 2.5

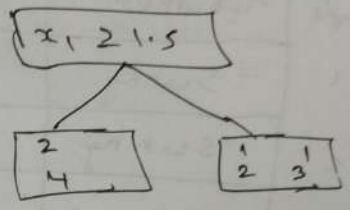
$x_2 = 2 \quad 2 \quad 2 \quad 3$

S.U = 2.5

$y = 3 \quad 4 \quad 6 \quad 7$

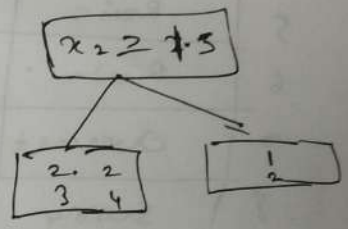
	S.U	M
x_1	1.5	4.68
	2.5	1

x_1	1.5	0.5
x_2	1.5	1.25



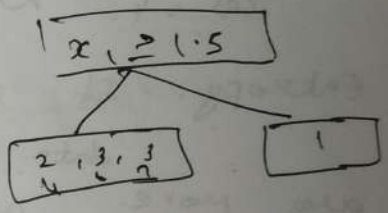
mean = 2.5

E = 0.5

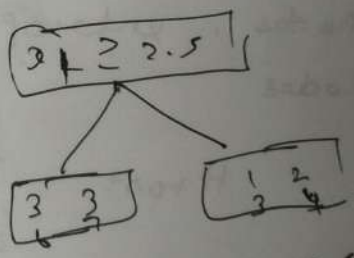


mean = 3.75

E = 1.25

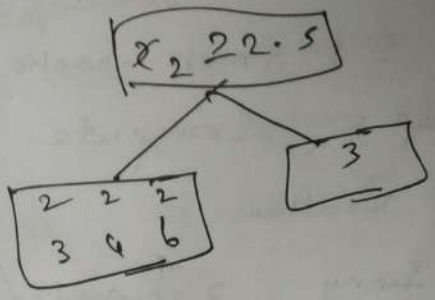


mean = 5.6



E = 0.5 mean = 3.5

E = 0.5 E = 0.5



$2.2^2 + 1.33^2$

16/02/2026. for classification only [illegible] function will change instead of ~~log~~ sum of squared error

Sample	Weather	Windy	Play Tennis → target variable
1	Sunny	No	No
2	Sunny	Yes	No
3	Overcast	No	Yes
4	Rain	No	Yes
5	Rain	Yes	No
6	Rain	No	Yes
7	Overcast	Yes	Yes
8	Sunny	No	No

Step 1: Compute root entropy for target variable

Yes = 4 No = 4 $P(Yes) = \frac{4}{8} = 0.5$ $P(No) = \frac{4}{8} = 0.5$

Entropy: If entropy of a node is very high, the data is impure, randomness is more, classes are more and predictability is difficult, also called homogeneous and heterogeneous nodes, entropy will be 0 at homogeneous nodes

Shanon's entropy

$$H_{root} = \sum_{i=1}^{2} -P_i \log_2 P_i$$ ⇒ minimal bytes required to transfer data from A to B

$$= -P_{yes} \log_2 P_{yes} + (-P_{No} \log_2 P_{No})$$

$$= -0.5 \log_2 (0.5) + (-0.5 \log_2 (0.5))$$

$= 1$ ⇒ more chaotic

i is the no. of classes in target variable

Step 2: Compute Information Gain, for feature = weather for weather

Sunny : 3 samples

Overcast : S_3 , S_7 $\quad P(Yes) = 1$

Yes Yes

using Shanon's Entropy $H = 0$

Rain : S_4 S_5 S_6 $\quad P(No) = \frac{1}{3}$ $\quad P(Yes) = \frac{2}{3}$

Yes No Yes

$$H = -\frac{1}{3}\log_2\left(\frac{1}{3}\right) - \frac{2}{3}\log_2\left(\frac{2}{3}\right)$$

$$= -\frac{1}{3}(-1.58) - \frac{2}{3}(-0.58)$$

$$= 0.526 + 0.386$$

$$= 0.912$$

Step 3:

Compute weighted entropy

$$H_{weighted} = \frac{\text{No. of. Samples}}{\text{Total samples}} H_{child} \quad (\text{Summation of all})$$

$$= \frac{3}{8} \times (0) + \frac{2}{8} \times (0) + \frac{3}{8} \times (0.912)$$

$$= 0.342$$

$$IG_{weather} = H_{parent} - H_{child\ (weighted)}$$

$$= 1 - 0.342$$

$$= 0.658$$

(IG is opposite of entropy intuitively)

Calculate IG for all features and pick the feature with maximum IG

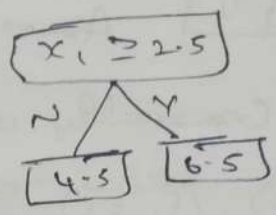
Max voting procedure => for interference in a leaf node values of that particular ... max number of

Example.

x_1	x_2	y
1	2	4
2	3	5
3	4	6
4	5	7

Iteration 1

$r_1 = y_1$ a) fit model F_1



$\lambda = 0.1$

$r^1 = r^1 - \lambda f_1(x^{(1)})$

$= 4 - (0.1 \times 4.5)$

$= 3.55$

$r_2 = 5 - 0.45$

$r_2 = 4.55$

$r_3 = 6 - 0.65$

$r_3 = 5.35$

$r_4 = 7 - 0.65$

$r_4 = 6.35$

$\therefore r = \begin{bmatrix} 3.55 \\ 4.55 \\ 5.35 \\ 6.35 \end{bmatrix}_{4 \times 1}$

Iteration 2

Dataset becomes

x_1	x_2	r
1	2	3.55
2	3	4.55
3	4	5.35
4	5	6.35

r value

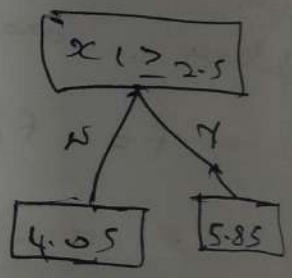
$r^1 = 3.55 - 0.405$

$r^1 = 3.145$

$r^2 = 4.55 - 0.45$

$r^2 = 4.145$

assume.



$r^3 = 5.35 - 0.585$

$r^3 = 4.765$

$r^4 = 6.35 - 0.585$

$r^4 = 5.765$

$r = \begin{bmatrix} 3.145 \\ 4.145 \\ 4.765 \\ 5.765 \end{bmatrix}$

Boosting

i) Adaboost for classification

ii) Gradient boosting for classification

iii) Gradient boosting for regression

currently practiced

03/03/2026 Adaboost

Setting i) Binary classification $(-1, +1)$

ii) Train B models $f_b \longrightarrow \{-1, +1\}$

iii) Combine weighted votes

final output

$F(x) = \text{sign}\left[\sum_{b=1}^{B} \lambda_b f_b(x; \theta_b)\right]$ → λ is changing for all models

sign (-3.45):

1. Initialize training sample weights

$w^{(i)} = \frac{1}{N}$ for all N samples

2. for iteration $b = 1$ to B

{ a) fit a model $f_b(x; \theta_b)$ such that it minimizes classification error weighted by $w^{(i)}$

b) set the model weight using error ε

$$\lambda_b = \frac{1}{2} \log\left(\frac{1-\varepsilon}{\varepsilon}\right)$$

c) update sample weights $w^{(i)} = w^{(i)} e^{-\lambda_b}$, if $f_b(x^{(i)}; \theta_b)$ is correct

$w^{(i)} = w^{(i)} e^{\lambda_b}$; if $f_b(x^{(i)}; \theta_b)$ is incorrect

}

If $x < 0$, $0 < e^{-x} < 1$

If $x = 0$, $e^{x} = 1$

If $x > 0$, $e^{x} > 1$

$$\varepsilon = \frac{\sum_{i=1}^{N} w^{(i)} I\{y^{(i)} \neq f_b(x; \theta_b)\}}{\sum_{i=1}^{N} w^{(i)}}$$

I => indicator function

It will return $\boxed{1}$ if condition is true else $\boxed{0}$

Normal entropy (weighted by $w^{(i)}$)

$$H = -\sum_{k=1}^{N} p_k \log_2 p_k$$

$$p_k = \frac{\sum_{i \in \text{class } k} w_i}{\sum w_i}$$

brought in during decision stump construction

for every split value

$$f_t = \underset{t}{\operatorname{argmin}} \sum_i w^{(i)} I(y^{(i)} \neq f(x^{i}; \theta))$$

$0 \le x_1 \le 6$

Ex:

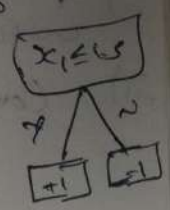
x_1	x_2	y	w^i
1	2	+1	0.25
2	1	-1	0.25
4	4	+1	0.25
5	3	-1	0.25

for x_1 split value 1.5, 3, 4.5

for 1.5, $\varepsilon = 0.25$

for 3, $\varepsilon = 0 + 0.25 + 0 + 0.25$
$= 0.5$

for 4.5, $\varepsilon = 0 + 0.25 + 0 + 0$
$= 0.25$



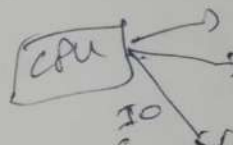
for x_2

$x_2 \le 4.5$

Parallel

for finding split values, we need to sort samples, that sorting is done in multiple computers for same data.

Cache - Aware - Access:



needs more frequently store in cache

Input output Access, amount of time required to transfer data from HDD to CPU memory is very high.

∴ Time CPU memory < main memory < hard disk

Compress the data and store in HDD. and bring back to main and do decompression

Block for out of core computation:

every node => 1 subset of data

from all different CPU data comes parallelly and processed.

Sharding:

distributing data across multiple databases

Light GBM (odds, Ex. IND vs AUS, 10M, 8 won, log odds = $\frac{8}{2} = 4$ (won / lost))

11/03/2026 ~~Kaggle~~ ~~methods~~ Gradient Boosting for classification

Chest Pain	Good Blood Circulation	Blocked arteries	Heart disease	Residual
No	No	No	No (0)	0-0.66
Yes	Yes	Yes	Yes (1)	1-0.66
Yes	Yes	No	Yes (1)	1-0.66
Yes	Yes	No	Yes (1)	1-0.66
Yes	No	Yes	Yes (1)	1-0.66
No	No	No	No (0)	0-0.66
	Yes			

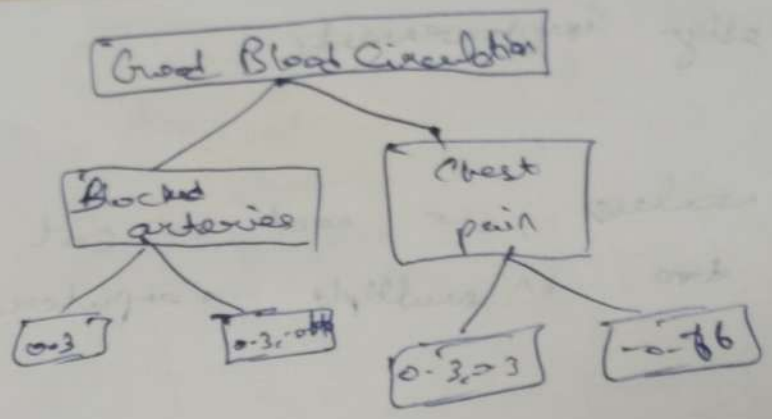
1) log-odds
2) convert to probability.

Step 1 : Initial Prediction

$\log(\text{odds}) = \log\left(\frac{4}{2}\right) \approx 0.7$

convert this to probability

$\log(\text{odds}) \quad \log\left(\frac{4}{2}\right) = 0.66$



Affine ai/gatin
boosting
trees for classifica

⇒ Predict Residuals and convert the leaf values to log odds

$$= \frac{\Sigma_i \text{ Residual}_i}{\Sigma_i \left[\text{Previous prob}_i \times (1 - \text{prev prob}_i)\right]}$$

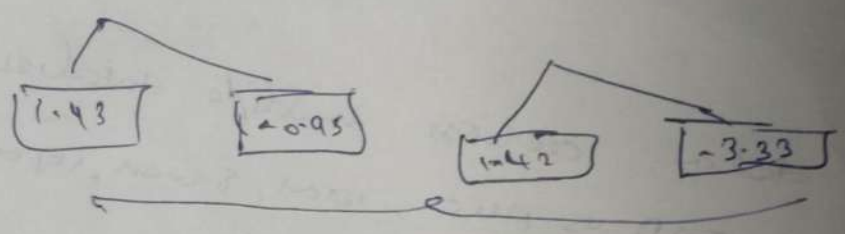
for 1st leaf :- $\frac{0.3}{0.66 \times (1-0.66)}$

$= 1.43$

for 2nd leaf :- $\frac{0.3 + (-0.7)}{(0.66(1-0.66)) + (0.66(1-0.66))}$

$= -0.89$

Now the tree leaves will be modified to



Log-odds value ⇒ convert to probability then we previous residual value subtract and pass it to next model.

12/07/2026

Kernel methods

Lower dimensional data to higher dimensional

transform (charge)

$x = \begin{bmatrix} 1 \\ 2 \end{bmatrix}_{2\times 1}, \quad z = \begin{bmatrix} 3 \\ 4 \end{bmatrix}_{2\times 1} \quad R^2$

Kernel function

$$\phi(x) = \begin{bmatrix} x_1^2 \\ x_2^2 \\ \sqrt{2}\, x_1 x_2 \\ \sqrt{2}\, x_1 \\ \sqrt{2}\, x_2 \\ 1 \end{bmatrix}_{6\times 1} \quad R^6$$

$K(x, z) = (x \cdot z + 1)^2$

$(x \cdot z + 1)^2 = (3 + 8 + 1)^2$

$= (12)^2$

$= 144$

$\phi : R^2 \to R^6$

$x = \begin{bmatrix} 1 \\ 4 \\ 2\sqrt{2} \\ \sqrt{2} \\ 2\sqrt{2} \\ 1 \end{bmatrix} \quad z = \begin{bmatrix} 9 \\ 16 \\ 12\sqrt{2} \\ 3\sqrt{2} \\ 4\sqrt{2} \\ 1 \end{bmatrix}$

$\phi(x) \cdot \phi(z) = 9 + 64 + 48 + 6 + 16 + 1$

$= 144$ Hence proved again

1) ~~Fisher~~ Linear kernel : $K(x, z) = x \cdot z$

2) Polynomial kernel : $K(x, z) = (x \cdot z + c)^d$

3) Fisher kernel

4) Graph kernel

5) $\boxed{\text{Radial Basis function (RBF) kernel}}$ => main focus

6) String kernel

7) Neural Network gaussian process (NNGP) kernel

Claim : Principles of mathematic induction => ways to prove theorem

$$\theta = \sum_{i=1}^{n} \beta_i \, \phi(x^{(i)})$$

Assume $\theta = 0$

Gradient descent update rule

θ^{t+1}

$a^T b = x a^T a$ (x is scalar)

$x = \frac{a^T b}{a^T a}$ → (gives a^2 which is a scalar hence division is possible)

$\boxed{p = a \frac{a^T b}{a^T a}}$ $p = PM\, b$ PM :) projection matrix

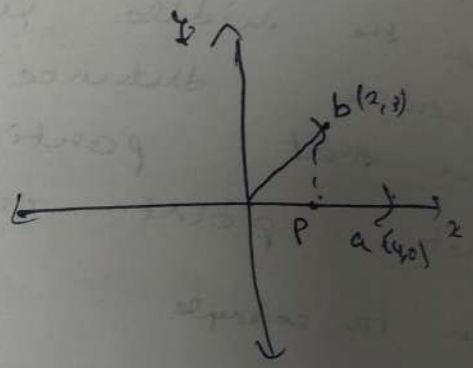
$\boxed{PM = \frac{a a^T}{a^T a}}$ projection matrix is 1g

also $PM = a(a^T a)^{-1} a^T$

$a a^T$ is a vector
$a^T a$ is a scalar

Ex: $b = \begin{bmatrix} 2 \\ 3 \end{bmatrix}$ $a = \begin{bmatrix} 4 \\ 0 \end{bmatrix}$ ⇒ horizontal line along x axis

project b onto a. find p



$$PM = \frac{\begin{bmatrix} 4 \\ 0 \end{bmatrix} \begin{bmatrix} 4 & 0 \end{bmatrix}}{\begin{bmatrix} 4 & 0 \end{bmatrix} \begin{bmatrix} 4 \\ 0 \end{bmatrix}}$$

$$= \frac{\begin{bmatrix} 16 & 0 \\ 0 & 0 \end{bmatrix}}{16}$$

$$= \begin{bmatrix} 1 & 0 \\ 0 & 0 \end{bmatrix}$$

$p = PM \times b$

$$= \begin{bmatrix} 1 & 0 \\ 0 & 0 \end{bmatrix} \begin{bmatrix} 2 \\ 3 \end{bmatrix}$$

$$= \begin{bmatrix} 2 \\ 0 \end{bmatrix}$$

to [illegible] curve (AUC) or AUROC

in class bias, where eg in 1:100 1:95 (+ve), 1:5 (-ve)

more accuracy (90%). least F_1 ✗ not good

AUPRC → average precision

Precision (Y) recall (X)

Regression - metrics

$$R^2 = 1 - \frac{RSS}{TSS} = 1 - \frac{\sum_{i=1}^{N} (y_i - \hat{y}_i)^2}{\sum_{i=1}^{N} (y_i - \bar{y}_i)^2}$$ higher value better fit.

coefficient of determination

proportion of the variation in the dependent variable (target) that is predictable from the independent variable (features).

represents proportion of variance explained by the model.

$\bar{y}_i$ ⇒ mean of ground truth y.

False Positive Rate = 1 - specificity.

True Positive Rate = sensitivity.

$$MSE = \frac{1}{N} \sum_{i=1}^{N} (y_i - \hat{y}_i)^2$$

$$RMSE = \sqrt{MSE}$$

$$\text{mean absolute error} = \frac{1}{N} \sum_{i=1}^{N} |y_i - \hat{y}_i|$$

$$\text{mean absolute \% error} = \frac{1}{N} \sum_{i=1}^{N} \left| \frac{y_i - \hat{y}_i}{y_i} \right|$$

Test data

Actual label	Predicted score	Predicted label (0.5)	(0.7)	(0.4)
1	0.85	1	1	1
0	0.6	1	0	1
1	0.7	1	1	1
1	0.4	0	0	0
0	0.55	1	0	1
1	0.5	0	0	1
0	0.65	1	0	1
1	0.35	0	0	0
0	[illegible]	1	0	1

threshold > 0.5

3	3
2	2

A = 0.5

P = 0.5

S = 0.6

specificity

FN FP TP

TN FN FP

FN FP FP

FN FN FN

FP FN FP

FP FN FP

FP FN FP

FN FN FP

Eigen values and Eigenvectors.

$$A = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}$$

$$\begin{vmatrix} 2-\lambda & 1 \\ 1 & 2-\lambda \end{vmatrix} = 0.$$

$$4 + \lambda^2 - 4\lambda - 1 = 0$$

$$\lambda^2 - 4\lambda + 3 = 0.$$

$$\lambda^2 - 3\lambda - \lambda + 3 = 0.$$

$$\lambda(\lambda - 3) - 1(\lambda - 3) = 0$$

$$\boxed{\lambda = 3 \text{ and } \lambda = 1.}$$

for $\lambda = 3$

$$\begin{pmatrix} -1 & 1 \\ 1 & -1 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} = 0$$

$-x_1 + x_2 = 0$ One eigen vector, $x_1 = x_2$

$x_1 - x_2 = 0.$ Let $x_1 = 1$, $x_2 = 1$

Eigen vector $\begin{pmatrix} 1 \\ 1 \end{pmatrix}$

$$\begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} = 0.$$

$$x_1 + x_2 = 0$$

$x_1 = -x_2$ Other eigen vector $\begin{pmatrix} 1 \\ -1 \end{pmatrix}$

PCA

$$X = \begin{bmatrix} 2 & 0 & 1 \\ 0 & 2 & 1 \\ 3 & 1 & 2 \\ 1 & 3 & 2 \end{bmatrix}$$

Step 1: Mean centering.

$\mu = [1.5, 1.5, 1.5]$

$$\therefore X_c = \begin{bmatrix} 0.5 & -1.5 & -0.5 \\ -1.5 & 0.5 & -0.5 \\ 1.5 & -0.5 & 0.5 \\ -0.5 & 1.5 & 0.5 \end{bmatrix}$$

centered.

Step 2: Covariance matrix.

$$\begin{bmatrix} cov(x_1,x_1) & cov(x_1,x_2) & cov(x_1,x_3) \\ cov(x_2,x_1) & cov(x_2,x_2) & cov(x_2,x_3) \\ cov(x_3,x_1) & cov(x_3,x_2) & cov(x_3,x_3) \end{bmatrix}$$

Covariance matrix $= \frac{1}{n} X_c^T X_c$

$$C = \begin{bmatrix} 1.25 & -0.75 & -0.5 \\ -1.75 & 1.25 & 0.5 \\ -0.5 & 0.5 & 0.25 \end{bmatrix}$$

$\lambda_1 \approx 2.0$, $\lambda_2 \approx 0.75$, $\lambda_3 \approx 0$ $\Leftarrow$ Eigen values

$u_1 = \frac{1}{\sqrt{2}}[1, -1, 0]$

$u_2 = \frac{1}{\sqrt{6}}\begin{bmatrix} 1 \\ 1 \\ 2 \end{bmatrix}$

$u_3 = \frac{1}{\sqrt{3}}\begin{bmatrix} -1 \\ 1 \\ 1 \end{bmatrix}$

Dimensionality reduction!

3D $\rightarrow$ 2D

Top 2 eigen vectors

$W_{2D} = [u_1, u_2]$

Project data points, $Z = X_c W_{2D}$

Now each point will be 2D. variance explained by eigen values

2D $\rightarrow$ 1D Take top 1 eigen vector $W_{1D} = [u_1]$

Total variance $= 2 + 0.75 + 0$

$= 2.75$

$PC_1 = \frac{2}{2.75} \approx 73\%$, $PC_3 = 0\%$

$PC_2 = \frac{0.75}{2.75} \approx 27\%$

3D $\rightarrow$ 2D gives 100% variance, 3D $\rightarrow$ 1D given only 73% variance

13/04/2026

1) Visualization

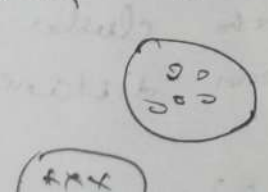
2) Grouping or clustering

i) K-means

ii) Hierarchical

* Asslomerative

K-means consider group of samples.



Notations:

n samples - $x^{(i)}$, $i \in 1$ to N. $x \in R^d$

denote the sets containing indices of

for cluster C_K

measure $W(C_K)$

- is the amount by which observations or samples differ from each other

- we need to solve

$$\underset{C_1, C_2, \ldots C_K}{\operatorname{argmin}} \left\{ \sum_{k=1}^{K} W(C_K) \right\}$$

all pairwise comparisons

$$W(C_K) = \frac{1}{|C_K|} \sum_{i, i' \in C_K} \sum_{j=1}^{d} \left(x_j^{(i)} - x_j^{(i')} \right)^2$$

where $|C_K|$ denotes the number of observations in cluster K

$$\min_{C_1, C_2, \ldots C_K} \left\{ \sum_{i=1}^{K} \frac{1}{C_K} \sum_{i, i' \in C_K} \sum_{j=1}^{d} \left(x_j^{(i)} - x_j^{(i')} \right)^2 \right\}$$

$\Rightarrow$ objective function

$\Rightarrow$ intractable

exponential

greedy

Greedy algorithm for K-means

1) Randomly assign a number from 1 to K for each observation / sample

2) Iterate until the cluster assignments stop chang

a) For each of the K clusters, compute the cluster centroid, the Kth cluster centroid a vector of d feature means for the observ in the Kth cluster.

b) Assign each observation / sample to the cluster whose centroid is closest by Euclidean distance

End

	x_1	x_2
S_1	1	1
S_2	1	2
S_3	2	2
S_4	8	8
S_5	8	9
S_6	9	8

$K = 2$

$K = 1$

1	1
1	2
2	2

Centroid = $\left(\frac{4}{3}, \frac{5}{3}\right)$

$K = 2$

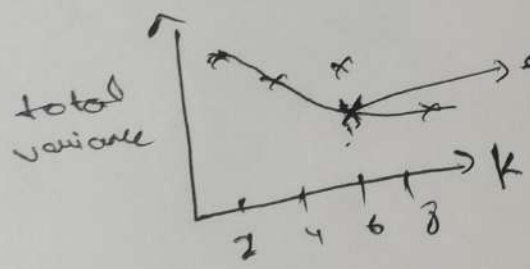
8	8
8	9
9	8

16/04/2026 Calculate distance from each point to centroid

	$K=1$	$K=2$
$S_1 \Rightarrow$	$\sqrt{\left(\frac{1}{3}\right)^2 + \left(\frac{2}{3}\right)^2} = 0.74$	$\sqrt{\left(\frac{22}{3}\right)^2 + \left(\frac{22}{3}\right)^2} = 10.37$
$S_2 =$	$\sqrt{\left(\frac{1}{3}\right)^2 + \left(\frac{1}{3}\right)^2} = 0.47$	$\sqrt{\left(\frac{22}{3}\right)^2 + \left(\frac{19}{3}\right)^2}$
$S_3 =$	$\sqrt{\left(\frac{2}{3}\right)^2 + \left(\frac{1}{3}\right)^2} = 0.74$	$\sqrt{\left(\frac{19}{3}\right)^2 + \left(\frac{19}{3}\right)^2}$
$S_4 =$	$\sqrt{\left(\frac{20}{3}\right)^2 + \left(\frac{19}{3}\right)^2}$	$\sqrt{\left(\frac{1}{3}\right)^2 + \left(\frac{1}{3}\right)^2} = 0.47$
$S_5 =$	$\sqrt{\left(\frac{20}{3}\right)^2 + \left(\frac{23}{3}\right)^2}$	$\sqrt{\left(\frac{1}{3}\right)^2 + \left(\frac{2}{3}\right)^2} = 0.74$
$S_6 =$	$\sqrt{\left(\frac{23}{3}\right)^2 + \left(\frac{19}{3}\right)^2}$	$\sqrt{\left(\frac{2}{3}\right)^2 + \left(\frac{1}{3}\right)^2} = 0.7$

Elbow method

fix K, for different K how total variance within cluster varies, biggest disadvantage: no relation between n and K, cannot choose K value



elbow point, point after which total variance does not decrease

Hierarchial clustering → no choosing K value
↓ visually interpretable
given dendrogram (trees turned upside down)